Coppice Names:
Private, Trustless Names on Zcash

Draft whitepaper, version 0.2
September 2026

Abstract

Coppice Names maps a short `.zec` name to a canonical Zcash Unified Address without adding a sidechain, consensus change, trusted name server, global registry operator, or protocol-specific token. Each registration is backed by an exact 1 ZEC shielded note that remains under its owner's control, and releasing a name is an ordinary spend that returns the bond, less the normal Zcash transaction fee.

The protocol combines a name-hiding `COMMIT`, a scheduled name-routed `REVEAL`, and a proof-carrying `REFRESH` operation. Zero-knowledge proofs establish hidden ownership and bond-note transitions, while a deterministic reducer applies public timing, ordering, lineage, and spentness rules against the canonical Zcash chain. Any wallet can resolve an arbitrary exact name by deriving its public route, authenticating the referenced historical `COMMIT`, and replaying the name's accepted lineage. A rolling cache of canonical compact Zcash evidence makes this process fast without turning a local index into an authority.

Coppice Names is implemented and exercised by conformance vectors, but it is not deployed. This document presents the design, its security and privacy analysis, and preliminary measured performance. It is an explanatory whitepaper, not the normative protocol definition; implementations must follow the protocol specification and its conformance artifacts.

Status of this document

Coppice Names is implemented but not deployed. This document presents the intended protocol as one coherent design and explains the reasoning behind its mechanisms. It is an explanatory whitepaper, not the normative

protocol definition; implementations must follow the protocol specification and its conformance artifacts.

Area	Status
Protocol construction and deterministic reducer	Implemented
`REVEAL` and `REFRESH` circuits	Implemented; exercised by conformance vectors
Exact-name resolver and wallet integration	Implemented
Six-month replay performance study	Preliminary; measured and modeled as labelled
Independent cryptographic review	WIP
End-to-end deployment qualification	WIP
Production deployment parameters and activation	TBD

Statements marked **WIP**, **TBD**, **measured**, or **modeled** retain those qualifiers until the corresponding work is complete.

Contents

1 Introduction

1.1 The problem

Cryptocurrency payment addresses are precise but difficult for people to recognize, remember, and verify. A naming layer can make payments usable, but a conventional naming service usually buys usability by introducing something a privacy-oriented currency cannot accept: an operator who controls the registry, an externally trusted index that answers lookups, public ownership records that turn registrations into identity disclosures, or a separate consensus system that fragments security.

1.2 Why naming is hard on a privacy currency

A naming layer for a privacy currency faces a tension that a plain public ledger does not. Resolution by strangers requires that some public data bind a name to a payment destination. Privacy requires that ownership — the keys and notes that actually control the record and its bond — stay hidden. And trustlessness requires that no operator be able to censor, correlate, or falsify a lookup.

Each naive design sacrifices one side of this triangle:

- A *public on-chain registry* resolves cheaply but publishes who owns which name, when they update it, and when they move value.

- A *trusted resolver service* can hide records but becomes a surveillance and censorship point, and its answers are not protocol evidence.
- A *consensus-level registry* (a name state root committed by the chain) would make currentness derivable from consensus, but demands a network upgrade and couples name policy to consensus policy.
- A *mutable local index* is fast but is only a claim: nothing about a plausible database entry makes it a correct answer.

Coppice Names is designed for the narrow region in which none of these compromises is made. The chain keeps authority; proofs keep ownership hidden; deterministic replay keeps answers verifiable; and the schedule keeps resolution cheap enough that no third-party answer is ever needed.

1.3 Requirements

Coppice Names begins from five requirements:

1. A resolver must be able to resolve any exact name, not only names it previously followed.
2. Correctness must come from Zcash's canonical transaction history and locally verified protocol rules, not from a trusted third-party answer.
3. Registration authority and the bonded note should remain shielded.
4. The registrant must retain ownership of the bond; registration must neither burn ZEC nor transfer it to a protocol operator.
5. The construction must fit within current Zcash consensus and transaction capabilities.

The result is an application protocol carried by Zcash transactions. Zcash provides transaction validity, canonical ordering, shielded-note semantics, and fork choice [1, 2]. Coppice provides authenticated application publication and replay. Coppice Names adds name-specific proofs and deterministic state rules.

1.4 Non-goals

Coppice Names does not attempt to provide:

- ownership transfer between hidden authorities;
- fuzzy, prefix, or directory search as a consensus feature;
- permanent names without renewal;
- free transaction publication;
- a trusted remote resolver or hosted index; or
- a Names state root committed by Zcash consensus.

Wallets may build local search interfaces over derived state, but such indexes are conveniences rather than resolution authority.

1.5 Contributions

As implemented, the construction contributes:

1. an application-layer naming protocol on *unchanged* Zcash consensus, deployable by wallets without a network upgrade, sidechain, or token;
2. a hidden-authority bond lineage: two fixed Halo2 proofs establish ownership and exact 1 ZEC bond-note transitions while the spending authority and note plaintexts remain shielded, and the bond stays refundable;
3. deterministic name routing and scheduled windows that bound the work of resolving an arbitrary name without continuous observation of unrelated traffic;
4. an exact-name resolver specified to produce the same lifecycle, record, head, and producer as a complete replay at the same canonical tip; and
5. a reproducible assurance package: positive conformance vectors verified by independent Rust and Python consumers, negative-test requirements, and a measured six-month replay performance study.

2 Background

This section gives the minimum background needed to read the rest of the document. The protocol specification is normative.

2.1 The Zcash shielded model

Zcash transactions may contain *Orchard actions* [3]. Each action consumes one shielded note (revealing only a *nullifier*) and creates another (revealing only a *note commitment*). Spending authority over a note is held by a key that is never published; the chain can verify that a nullifier and a commitment are honestly derived, but cannot link the two sides or identify the owner. The note commitment tree accumulates commitments in canonical action order, and consensus finalizes which nullifiers have been spent. NU6.3 moves Orchard activity to the *Ironwood* transaction format [2, 6, 7]; Coppice is Ironwood-only.

A *Unified Address* (UA) [4] packages one or more receivers (Orchard, Sapling, transparent) behind a single string, letting the *payer's* wallet choose a receiver by its own policy. Zcash fees and block limits are governed by standard policy [5]; publication is never free.

2.2 The Coppice runtime

Coppice is a deterministic application runtime over canonical Zcash Ironwood history [9]. It is application-blind: Zcash remains the sole chain and fork-choice authority, and Coppice consumes one validated canonical scan to host native deterministic state machines. Applications publish *bulletins* inside ordinary Orchard transactions: a bulletin's bytes are carried by zero-value *carrier notes* addressed to a rendezvous receiver, framed by the CPCF carrier protocol and wrapped in a CAPP application envelope naming an exact content-addressed application identity. Observers detect candidates under the deployment's rendezvous incoming viewing key, fetch full transactions only for candidates, and Core authenticates those bytes against compact canonical effects before routing them to an application.

Two properties matter for Names. First, bulletins are ordinary shielded traffic to third parties: no consensus change, no Coppice-aware node, and no protocol token. Second, Core supplies authenticated per-action facts — nullifiers, note commitments, value commitments, and commitment-tree positions — that applications may consume as replay evidence.

Figure 1 shows the stack. Each layer is authoritative for exactly what it can know; nothing above invents consensus facts, and nothing below interprets application payloads.

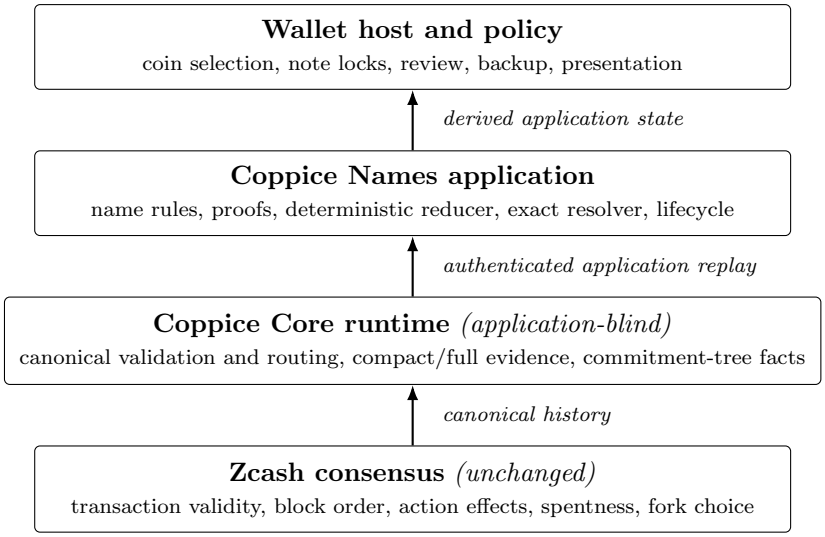

Figure 1: The Coppice Names layer stack. Zcash consensus is the sole transaction and fork-choice authority; Coppice authenticates application publication; Names adds name-specific validity; the wallet applies policy. Derived caches exist at every upper layer, and none of them is authority.

3 System and trust model

3.1 Roles

The system has four principal roles:

- A *registrant* controls a Zcash wallet, a hidden per-name authority, and an exact 1 ZEC Ironwood note used as the bond.
- A *resolver* derives public name routes and independently evaluates Names operations found in canonical Zcash history.
- A *wallet host* authenticates compact and full transaction data, maintains canonical scan state, handles reorganizations, and protects managed bond notes from accidental ordinary spending.
- The *Zcash network* validates and orders transactions and supplies the underlying shielded-note state transition.

3.2 Divided authority

Authority is deliberately divided, and each layer answers only the questions it is positioned to know:

Layer	Authoritative for
Zcash consensus	Transaction validity, block order, action effects, spentness, and fork choice
Names proofs	Hidden `COMMIT` openings, hidden authority, exact bond value, and predecessor/successor note relations
Names reducer	Operation timing, accepted lineage, uniqueness, lifecycle, and deterministic conflict resolution
Wallet policy	Coin selection, local note locks, transaction review, backup, and user presentation

Correspondingly, there are four *classes of evidence*, and every conforming resolver must apply all of them:

Evidence class	Establishes
Consensus-authenticated	Transaction identity, canonical order, action effects, spentness
Proof-established	Hidden openings, note ownership, exact bond relations
Replay-derived	Lifecycle, accepted lineage, window admissibility, currentness
Wallet policy	Note locks, presentation, recovery behavior — never protocol truth

A Zcash-valid transaction is not automatically an accepted Names operation. Likewise, a valid Names proof does not establish that its operation is current, timely, correctly routed, or canonically first.

3.3 Threat model

The construction assumes the security of the Zcash consensus rules it uses, the soundness and zero-knowledge properties of the proof system selected by a deployment, and the collision and preimage resistance of its domain-separated hash functions. A consensus-level adversary, or an adversary who breaks the proof system, is outside the application layer's scope.

Within that boundary, the analysis is organized around six adversaries:

Adversary	Capability and objective
A1: record attacker	Publishes valid Zcash transactions; wants to install or alter a record for a name it does not control. Cannot spend the owner's bond note.
A2: stale-state attacker	Replays previously valid protocol data — old proofs, old references, old branches — to redirect resolution or extend a lease.
A3: chain-data provider	Serves wallet chain data; may delay, omit, correlate, censor, or serve plausible lies. Must not be able to manufacture a Names result.
A4: window flooder	Fills one name's operation window with junk to raise resolution cost or grief its owner.
A5: linking observer	Watches public traffic and route activity; tries to link names, owners, resolutions, and payments.
A6: reorganizing minority	Produces short replacement branches consistent with consensus rules.

The corresponding security goals are: **G1** unforgeable authority (only the current hidden owner can change or release a record); **G2** freshness (only canonically current state can be extended or resolved); **G3** self-verifiable resolution (correctness does not depend on provider honesty); **G4** bounded work (per-name resolution cost is bounded under adversarial load); and **G5** limited disclosure (ownership stays hidden; lookup interest is minimized). Section 10 maps each adversary to the mechanisms that defeat it and the residual risk that remains.

A chain-data provider may delay, omit, correlate, or censor requests. It must not be allowed to manufacture a Names result merely by returning a mutable index entry. Availability and eclipse resistance inherit the wallet's chain source model; Coppice Names does not independently solve those network-level problems.

4 Protocol overview

4.1 Names and records

A canonical name is a lowercase ASCII label of 1 to 63 bytes containing letters, digits, and interior hyphens; it does not begin or end with a hyphen and contains no dot. The `.zec` suffix is presentation syntax and is not part of the protocol label. Each accepted record is a canonical ZIP-316 Unified Address for the deployment's Zcash network [4]. Any canonical receiver composition is allowed, including a UA containing a transparent receiver;

the paying wallet applies its own receiver-selection policy.

4.2 Deployment identity

Each deployment fixes its activation height, schedule, bond value, data bounds, application and runtime identities, and verifier identities. These values are hashed into a *deployment identifier*, preventing messages or proofs from one deployment from being replayed into another. The stable family label is `coppice.names`; the deployment identifier is included in the CAPP `ApplicationId`, and bulletins use Coppice CPCF framing [9]. That content-addressed identity selects the exact wire, proof, parameter, and reducer interpretation without a sequential protocol version or a separate consensus system.

4.3 Operations

The public protocol operations are:

Operation	Purpose	Name disclosed?	Bond effect
`COMMIT`	Commit to a name, target epoch, owner, and secret	No	None
`REVEAL`	Open a mature `COMMIT` and create a registered head	Yes	Creates the managed 1 ZEC state note
`REFRESH`	Update the UA and renew the lease	Yes	Spends and recreates the 1 ZEC state note
Ordinary spend	Release the name	No new bulletin; linkable through the current head	Returns control of the bonded value to the wallet

Figure 2 summarizes the lifecycle. `COMMIT` does not reserve a name. If competing registrations target the same reusable name, canonical block and transaction order determine which valid `REVEAL` is accepted first.

4.4 Design principles

Five principles recur throughout the construction, and most later sections are instances of them:

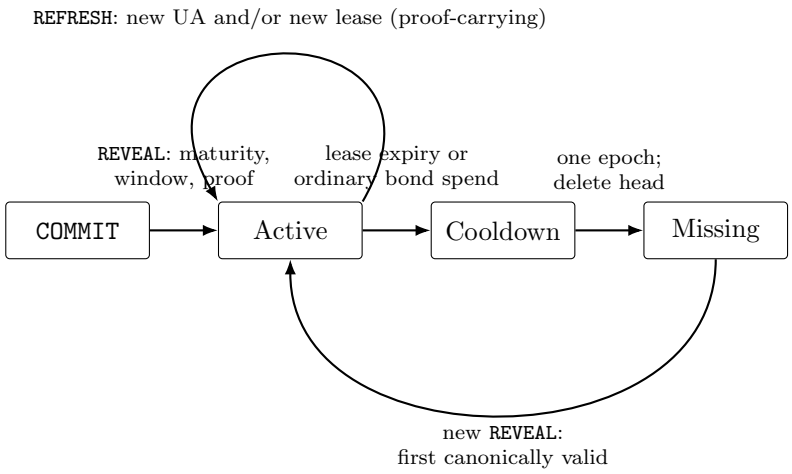

Figure 2: Coppice Names lifecycle. Acceptance of a `REVEAL` also requires the name window and a valid proof. A `COMMIT` is not a reservation. The former owner receives no special reclaim priority.

1. **Zcash stays the sole authority.** Consensus already decides validity, ordering, and spentness; Names never re-decides them.
2. **Proofs prove private relations only.** What arises from the chain — order, currentness, schedule — is checked publicly, not inside a circuit.
3. **All public rules are deterministic.** Two honest resolvers with the same canonical data must agree on every lifecycle decision.
4. **Cheap checks precede expensive proofs.** Framing, routing, scheduling, references, shapes, and lineage are rejected before any proof verifier runs.
5. **Derived caches are never authority.** Local indexes and snapshots accelerate replay; they cannot overrule authenticated history.

5 Registration: commit, reveal, and the bond

5.1 Commit–reveal

`COMMIT`s use the deployment's generic Coppice rendezvous route. They reveal a commitment but not the target name. `REVEAL` and `REFRESH` use a deterministic public route derived from the deployment identifier and the

name identifier. Anyone who knows a name can derive the same route and inspect only the blocks in which that name is eligible to operate.

This asymmetry serves two purposes:

- the initial commitment does not announce the target name before the reveal; and
- exact resolution does not require continuous acquisition of unrelated generic-rendezvous traffic.

The commitment opens to the deployment identifier, the name identifier, a target epoch, a hidden owner commitment, and a nonzero secret. A `REVEAL` carries an exact canonical reference to its historical `COMMIT`. The resolver fetches that one referenced compact transaction, authenticates the corresponding full transaction and Ironwood effects, and admits the `COMMIT` atomically with the `REVEAL` block. A location supplied by an untrusted source is never sufficient by itself. Figure 3 shows the timing.

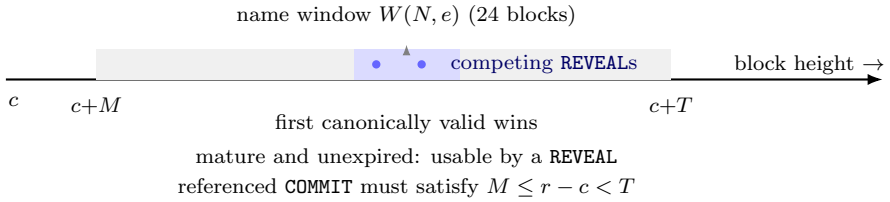

Figure 3: Commit–reveal timing (candidate production values: maturity $M = 24$ blocks, TTL $T = 192$ blocks, window $W = 24$ blocks). A `COMMIT` included at c becomes usable at maturity and is pruned at its TTL. Two valid `REVEAL`s in the window are resolved by canonical order, not by commitment order.

5.2 The bond

Names bulletins use Coppice application framing in zero-valued publication carriers. The bond is a separate *designated* Ironwood action and is never held or burned by a carrier. Registration creates a 1 ZEC shielded note controlled by the registrant's hidden authority; that note is the record's continuing existence. Figure 4 traces a note through its lineage.

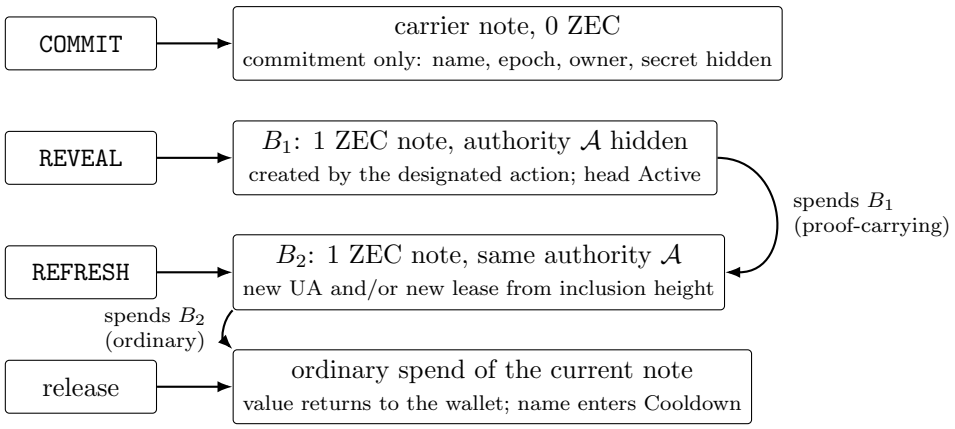

Figure 4: The bond-note lineage. Each accepted operation spends the current bond note and creates its successor with exactly the same hidden authority and exactly 1 ZEC. The head is whichever note the reducer currently accepts; release is an ordinary spend of that note, so it requires the owner's spending authority and is publicly detectable through the published future nullifier.

5.3 Why each mechanism exists

- *Commit–reveal* prevents mempool front-running of a chosen name: an observer of a `COMMIT` learns neither the name nor the owner, and a copied `REVEAL` cannot substitute a different hidden owner or bond note without satisfying the bound proof.
- *Canonical ordering instead of auctions or rents* keeps registration an ordinary publication: there is no operator to collect value, and no second market for allocation rights. Competing claims are decided by exactly the rule Zcash already provides — which valid transaction appears first in the canonical chain.
- *A refundable bond* makes names costly to hold at scale without burning value or introducing rent. The bond is the record: spending it is releasing it, so custody of the record and custody of the value are the same capability.

6 Schedule, renewal, and release

6.1 Name windows

For activation height A, epoch length E, name window length W, deployment identifier D, and name identifier N, the protocol derives one offset:

$$\begin{aligned} H_N &= \text{BLAKE2b-256}_{\texttt{CoppiceNmOff}}(D \parallel N), \\ \text{offset}(N) &= \text{LE64}(H_N[0..8]) \bmod (E - W + 1), \\ \text{window}(N, e) &= [A + eE + \text{offset}(N),\ A + eE + \text{offset}(N) + W). \end{aligned}$$

Intervals are half-open. `REVEAL` and `REFRESH` are accepted only inside the name's window; a referenced `COMMIT` must have reached its maturity and must not have reached its TTL. Cheap schedule and reference checks occur before proof verification. Figure 5 shows one epoch.

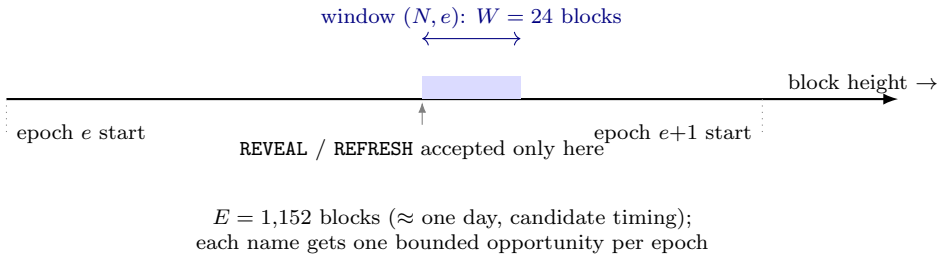

Figure 5: The name window. The offset is derived deterministically from the deployment and name identifiers, so every resolver computes the same window without any secret or service. A missed window costs at most one epoch of delay.

The current candidate profiles are:

Parameter	Production candidate	Regtest
Epoch	1,152 blocks	32 blocks
Name window	24 blocks	4 blocks
`COMMIT` maturity	24 blocks	4 blocks
`COMMIT` TTL	192 blocks	24 blocks
Lease	250,000 blocks	128 blocks
Cooldown	1,152 blocks	32 blocks

The production profile provides approximately daily operation opportunities and a roughly six-month lease under the candidate timing assumption.

Regtest is intentionally accelerated and is cryptographically separated by its deployment identifier. Production values remain **TBD** until deployment. Valid schedules satisfy $0 < W \leq M < T < E < L$ and cooldown $= E$.

6.2 REFRESH: update and renewal in one operation

`REFRESH` is deliberately a single operation. Changing the UA and renewing without changing it are the same protocol event: the prover demonstrates that the designated predecessor and successor notes both have value exactly 1 ZEC and are controlled by the same hidden authority, and the reducer separately requires the predecessor to be the exact current accepted head, spent by this transaction, inside the name's window, in a strictly later epoch than the predecessor's. Every accepted `REFRESH` starts a full new lease from its inclusion height. Transfer to a different hidden authority is not supported.

This design keeps the record's authority exactly as recoverable as the wallet's seed: one authority per name, derived from the seed, never rotated on-chain. It also removes a whole class of transfer-market and stolen-name problems by making transfer impossible at the protocol level.

6.3 Release, expiry, and cooldown

There is no `RELEASE` bulletin. The owner releases a name by making an ordinary valid Ironwood spend of the current hidden 1 ZEC bond; the reducer recognizes the canonical nullifier and the wallet normally returns the bond, minus the normal transaction fee, to the user's own control.

When a bond is spent or a lease expires, the name enters a uniform one-epoch cooldown during which it resolves to no payment address and no replacement can be installed — including by the former owner. This deliberately visible non-resolving interval reduces the risk that a recently terminated name immediately begins paying a new owner. The rule does not distinguish voluntary release from natural expiry, and the former owner receives no special reclaim priority. Cooldown is an anti-impersonation quarantine, not an ownership grace period: at the first height after cooldown, the reducer deletes the old head at block start and the first canonically ordered valid `REVEAL` may install a new head.

Resolution returns the UA only for the `Active` lifecycle. Cooldown may include head metadata for diagnostics but must not be presented as payable. Missing returns neither a head nor a UA and intentionally does not distinguish never-used from formerly-used names.

7 Hidden ownership: the zero-knowledge proofs

The registrant's per-name Orchard authority is never published. The protocol uses two fixed Halo2 proof statements, instantiated with an IPA polynomial commitment over the Pasta curves [8] with $k = 11$; each proof is 4,704 bytes. A deployment binds the generated verifying-key fingerprint, parameter exponent, operation tag, and fixed proof length into its verifier identity, so callers cannot select an alternate verifier.

7.1 What each proof establishes

REVEAL. The prover opens the referenced `COMMIT` and demonstrates that it binds the deployment, name, target epoch, hidden owner commitment, and a nonzero secret; and that the designated successor action creates a note controlled by that hidden authority with value exactly 1 ZEC. The public statement binds the canonical `COMMIT` reference, UA, inclusion epoch, action index, action nullifier and commitment, successor future nullifier, and bond value.

REFRESH. The prover demonstrates that the designated predecessor and successor notes both have value exactly 1 ZEC and are controlled by the same hidden authority. The statement binds the accepted predecessor reference and commitment, its future nullifier, the new canonical UA, the new epoch, and the successor action effects. Each Halo2 proof exposes one Pallas field: a Poseidon-folded digest of typed public facts reconstructed by replay, with length-framed, domain-separated byte inputs and distinct fold domains for the two operations. The exact statement layouts are frozen by the protocol specification and the conformance vectors.

7.2 What the proofs deliberately do not decide

The proofs establish private note relations; they do not replace canonical history. Ordering, schedules, currentness, competing spends, and reorganization handling remain deterministic public checks because those facts arise from the chain rather than from the registrant's private witness:

Question	Decided by	Evidence
Does this `REVEAL` open a real `COMMIT` binding deployment, name, epoch, owner, secret?	Proof	Private witness
Is the bond note exactly 1 ZEC and controlled by the hidden authority?	Proof	Private witness
Is the predecessor the exact current accepted head?	Reducer	Canonical order
Is inclusion inside the name's window?	Reducer	Schedule over canonical heights
Is the accepted head still unspent?	Reducer	Authenticated nullifiers

8 Canonical state and exact resolution

8.1 The reducer

Every resolver executes the same deterministic reducer over authenticated Zcash data, beginning immediately before the activation block. For each transaction it (1) records authenticated action nullifiers, (2) tries to apply the decoded Names operation, and (3) terminates any current head spent by that transaction even if its bulletin was missing, malformed, ambiguous, or invalid. Step 3 is the point: a proof-valid physical note can never silently replace accepted Names state, so the record lives in accepted lineage, not in whichever bond-like note happens to exist.

At block start, lease expiry is applied and terminal heads past cooldown are deleted. Transactions are then processed in canonical order; the first valid operation that changes a name can make later conflicting operations invalid. A `REVEAL` is accepted only when no retained head exists; a `REFRESH` only against the exact current `Active` head.

8.2 Resolving an arbitrary name

To resolve an arbitrary exact name, a client:

1. canonicalizes the label and derives its identifier, route, and eligible windows;

2. authenticates candidate `REVEAL` and `REFRESH` bulletins found in those windows;

3. follows a candidate `REVEAL`'s bounded reference to its one historical `COMMIT`;

4. applies proof, timing, routing, statement, and accepted-lineage checks; and

5. observes authenticated Ironwood nullifiers through the relevant canonical tail to determine whether the accepted head remains unspent and active.

An exact resolver and a complete replay must produce the same lifecycle, record, head, and producer at the same canonical tip. Figure 6 shows the pipeline.

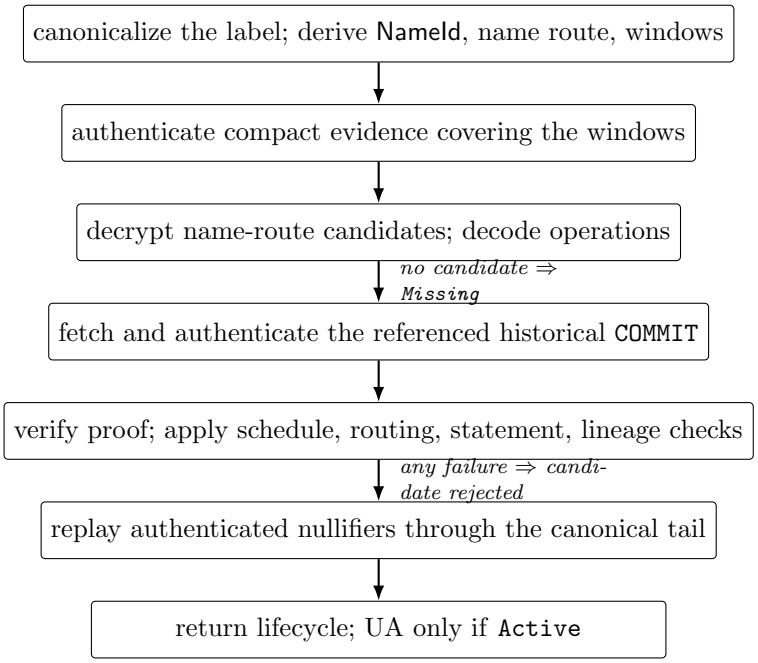

Figure 6: Exact resolution of one name. Every step uses authenticated evidence only; steps 1–3 use the deterministic schedule to touch as little of the chain as possible, and the final two steps restore the two facts that a schedule cannot shortcut: accepted lineage and current spentness.

8.3 Fast resolution without a trusted index

The protocol cannot derive currentness from a signed database that maps names to addresses: current Zcash consensus commits transaction history, not a Coppice Names state root. A mutable local index therefore cannot become authority simply because it records a plausible chain tip.

Instead, the wallet retains a rolling, branch-keyed window of authenticated compact Ironwood evidence as part of ordinary Zcash synchronization. Resolution still replays the deterministic rules locally, but it avoids downloading the same six-month history on demand. The evidence store is derived and reconstructible, follows normal wallet reorganization handling, and can be evicted beyond the protocol horizon.

The wallet's existing Ironwood commitment tree also supplies authenticated global action positions and tree-size transitions to Coppice. Names does not maintain a duplicate commitment tree. The host checks pre- and post-scan tree sizes and publishes candidate Names state only after the wallet's database transaction commits.

Figure 7 contrasts the two acquisition strategies. The generic rendezvous route is a shared firehose: observing it continuously would couple a resolver to every application's traffic. The name route is per-name: work is proportional to the requested name's windows alone, plus one referenced `COMMIT` fetch per candidate.

This construction preserves arbitrary-name resolution and local verification while moving the expensive work to the synchronization path the wallet already performs. The principal performance result in Section 14 is architectural for exactly this reason.

9 Wallet custody and recovery

9.1 The bond is the user's

The bond remains an ordinary user-controlled shielded note. A supporting wallet marks the exact 1 ZEC state note as managed so ordinary coin selection does not spend it accidentally. That lock is wallet safety policy, not protocol custody: the owner can explicitly release the name by spending the note.

9.2 Deterministic recovery

The companion wallet policy derives, from the wallet seed:

- a Names master;

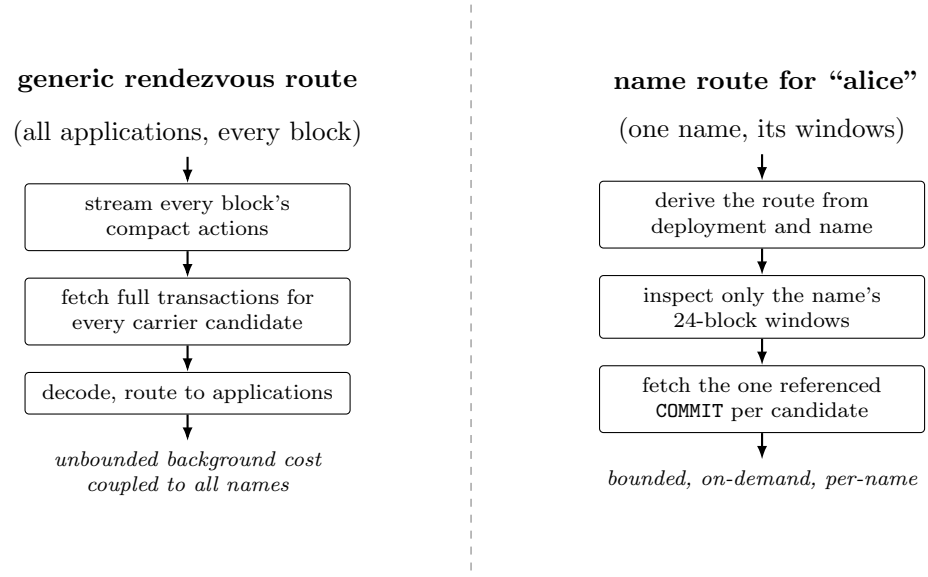

Figure 7: Publication routing asymmetry. `COMMIT`s hide on the generic route; `REVEAL`s and `REFRESH`es publish on the derived name route. Exact resolution therefore never acquires unrelated generic-route full transactions — the design that the adversarial-load analysis in Section 14 quantifies.

- a deployment- and name-separated hidden spending authority;
- deterministic epoch `COMMIT` secrets; and
- successor note material bound to canonical operation data.

The seed therefore recovers the secret authority. A nonsecret list of owned names tells a restored wallet which exact names to test and recover. No fragile sidecar secret is required, and importing one name does not require trusting a bulk registry snapshot. Figure 8 shows the derivation tree. Recovery is explicit wallet behavior: looking up an arbitrary name does not silently attempt ownership recovery, and a wallet may expose recovery even when other owned names are already present.

9.3 Transaction construction

Operations are constructed as version 6 (Ironwood) transactions through a PCZT pipeline, pinned to the NU6.3 branch, with the designated action pair carrying the bond spend and creation and carrier outputs pinned to the

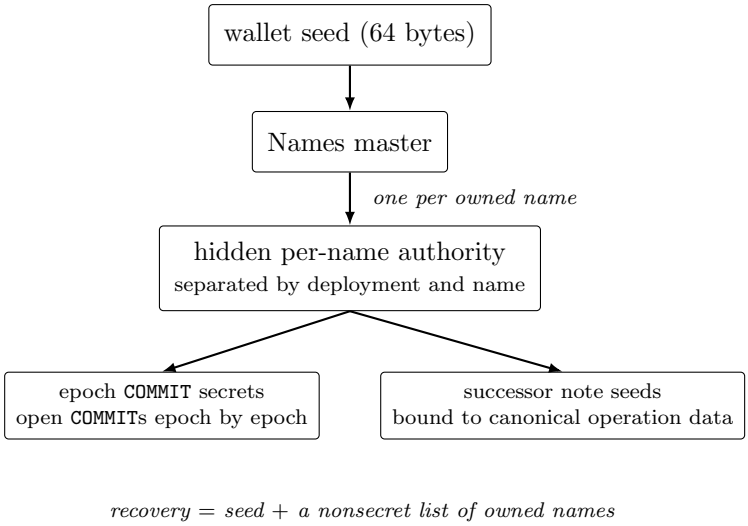

Figure 8: Wallet-side derivation (policy, not protocol validity). Every secret is derived from the seed, so the seed plus a plaintext list of owned names reconstructs the full lineage: the hidden authorities, the epoch `COMMIT` openings, and the deterministic bond-note material.

derived name route at zero value. The builder asserts that the designated input is exactly the current 1 ZEC bond and that the successor note's rho binds to the designated action's nullifier. These are wallet construction rules; the reducer independently re-derives the same facts from authenticated chain data.

10 Security analysis

This section maps each adversary from Section 3.3 to the mechanisms that defeat it and the residual risk that remains.

10.1 A1: record attacker (G1)

`COMMIT` hides the name and target opening before `REVEAL`. A copied `REVEAL` cannot substitute another hidden owner or successor note without satisfying the bound proof. A `COMMIT` does not reserve the name, so canonical ordering still decides between independently valid competing registrations after the old head is compacted.

A `REFRESH` must spend the exact current accepted predecessor and prove the same hidden authority controls its successor; an attacker cannot update the record using an older valid state. Release is the ordinary Zcash spend of the current bond note and therefore requires its spending authority. Residual risk: an attacker who obtains the owner's seed or device wins, as with any wallet.

10.2 A2: stale-state attacker (G2)

Canonical `StateRef` and `CommitRef` values bind height, transaction index, transaction identifier, and action index as applicable. The reducer accepts only the exact current predecessor, and a stale `REFRESH` cannot replace a newer head. Proof-valid notes on rejected or stale branches cannot become Names authority merely by existing on chain. Referenced `COMMIT`s admitted on demand are inserted atomically with the block containing their `REVEAL`, so on-demand evidence can neither linger after a failed block nor vanish on rollback.

10.3 A3: chain-data provider (G3)

A provider can degrade availability, correlate selective requests, or present the risks already present in the wallet's chain-source model. Coppice verifies transaction identities, action effects, route selection, references, chain continuity, and proofs before accepting an operation. A provider-supplied name-to-UA answer is never protocol evidence: resolution is computed by replaying authenticated data, so a lying answer is a denial of service, not a wrong address.

10.4 A4: window flooder (G4)

Candidate messages must pass inexpensive framing, route, schedule, reference, transaction-shape, and lineage checks before proof verification. Exact `REVEAL` discovery never requires fetching unrelated generic-route full transactions; only the historical `COMMIT` explicitly referenced by a candidate is acquired.

A targeted attacker can still publish traffic in a particular name's daily window. The window bounds the affected blocks, while every candidate must be published in a Zcash transaction, compete for block space, and ordinarily pay the applicable Zcash fee under relay and mining policy [5]. Coppice Names does not add transaction grinding: grinding complicates construction without removing the targeted publication bound. Section 14 quantifies the bounded cost.

10.5 A5: linking observer (G5)

The hidden Orchard authority and the ordinary shielded note plaintext remain hidden, and the `COMMIT` reveals neither name nor owner. What is intentionally public — the name, UA, schedule epoch, canonical references, and name-derived route after `REVEAL` — is analyzed in Section 11.

10.6 A6: reorganizing minority (G2)

Reducer snapshots and indexes are derived state. They are bound to deployment, network, height, and canonical block hash and are rolled back with the wallet's chain state. A block with the wrong height or previous hash is rejected. Deep reorganizations beyond retained journals require reconstruction from authenticated history rather than trusting the cached head.

10.7 Name impersonation after termination

Cooldown prevents immediate reassignment after either expiry or explicit release. During the interval the name resolves to no address, giving users and applications a deterministic warning gap before a later registrant can install a different record. It does not prove social identity and cannot prevent name squatting or user confusion after the cooldown ends.

10.8 Remaining assurance work

Formal security definitions, proof sketches for uniqueness and accepted-head agreement, an independent circuit review, and complete adversarial test coverage are **WIP**. No claim of a completed independent audit is made.

11 Privacy analysis

11.1 What is disclosed

A naming protocol must publish enough information for strangers to resolve a name. Table 1 states exactly what is public, when, and to whom.

11.2 Linkability and its limits

Operations for the same name are publicly linkable through the name and the predecessor chain. The designated state action is linked to the public

Information	Visible to	From
Per-name hidden authority	No one	Never; held by the owner's seed
`COMMIT` opening (name, epoch, owner, secret)	Owner only	Until `REVEAL`
Bond note plaintext and other wallet activity	Owner	Always, as in ordinary Zcash
Bond existence and exact 1 ZEC value	Public	From `REVEAL`
Name and UA	Public	From `REVEAL`
Operation lineage (`REFRESH` chain, windows, references)	Public	As published
Release event	Public (as an ordinary spend)	Detectable through the published head future nullifier
Resolution interest (which name a client looks up)	Remote provider, if any	Only for remote window queries; local evidence avoids it

Table 1: Disclosure matrix. Everything above the middle of the table is hidden by construction; everything below is necessarily public, because strangers must be able to resolve names and detect currentness without an operator.

Names operation: its proof establishes the exact 1 ZEC bond relation, and its future spend determines whether the name remains current. The authority and ordinary shielded note plaintext remain hidden, but the application-level lineage is intentionally observable. A UA may contain a transparent receiver because receiver composition is a record-owner choice, not a Names ownership signal; the protocol never requires a transparent output or transparent bond.

11.3 Lookup privacy

Remote exact-window requests can reveal which name a client is resolving. A rolling local evidence store reduces this leakage because normal wallet sync acquires the canonical evidence before the lookup and does not need a name-specific remote query. Private information can still leak through wallet network behavior, transaction timing, UA receiver composition, or compromise of the wallet itself.

Coppice Names offers privacy relative to public ownership registries and

trusted resolution services; it does not make public names or their payment records secret.

12 Economics and anti-spam

Every active name immobilizes exactly 1 ZEC in a shielded note controlled by the registrant. The value is refundable and is not a protocol fee. It imposes a capital cost on maintaining many simultaneous names while preserving the user's ownership of the funds.

`COMMIT`, `REVEAL`, `REFRESH`, and release are normal Zcash transactions to which the wallet applies Zcash fee policy. Those fees, relay policy, and Zcash block limits are the publication anti-spam mechanism; Coppice Names does not burn ZEC, levy rent, or create a separate fee market. The two costs play different roles and must not be conflated:

	Bond (1 ZEC)	Transaction fees
What it is	Refundable capital locked in the record's note	Cost of publishing an ordinary transaction
Where it goes	Back to the owner on release	Miners, under Zcash policy
What it prices	Holding a name active	Publishing an operation at all
Who sets it	Fixed by deployment identity	Zcash fee and relay policy

The wallet may display 1 ZEC as bonded rather than spendable while the name is active. An explicit release returns the note's value to ordinary wallet control minus the transaction fee. A fixed bond does limit accessibility as ZEC's purchasing power changes; that limitation is revisited in Section 16.

13 Related work

Naming systems occupy a design space spanned by three questions: does the system require its own consensus domain, who can see ownership, and what does holding a name cost. Table 2 positions Coppice Names against well-known systems at a conceptual level; each system's own documentation governs the details [10, 11, 12].

Three contrasts are worth drawing out.

Against public registries. Namecoin, ENS, and Handshake publish ownership and lineage by construction: registrations, updates, and transfers are

System	Consensus domain	Ownership	Cost to hold	Resolution trust	Transfer
Out-of-band UA sharing	Zcash (unchanged)	Private	None	Manual exchange	n/a
Namecoin	Own merge-mined chain	Public	Consumed fee plus renewal	Its own chain data	Yes
ENS	Ethereum contracts	Public (NFT)	Periodic rent, not refundable	Contract calls or gateways	Yes
Handshake	Own L1, auctions	Public	Auction commitments	Its own chain data	Yes
Coppice Names	None (Zcash application)	Hidden	Refundable 1 ZEC bond	Local replay; no operator	No

Table 2: Conceptual comparison. "Out-of-band UA sharing" is the status quo: fully private but with no trustless way for a stranger to resolve or verify a name. The three naming systems above all make ownership public and require their own consensus domain; Coppice Names is the only row that is both an application on an existing chain and hidden-authority.

public identity events. In a privacy currency, that is a cost, not a feature. Coppice Names inverts the design: the record (name, UA, schedule) is public, but the authority — the key material whose disclosure would let anyone else move or alter the record — is never published.

Against trusted resolution. Hosted resolvers and gateways can answer faster than local replay, but their answers are claims. Coppice Names keeps the verification local; speed is bought with derived local evidence (Section 8.3) rather than with trust.

Against consensus-level designs. A registry committed by consensus would make currentness cheap to prove, but it would require a network upgrade and would hard-code name policy into consensus. Coppice Names shows that the application layer can provide deterministic replay and bounded resolution on today's consensus, keeping name policy independent of Zcash's upgrade cycle.

14 Performance evaluation

The current evaluation uses a retained 250,000-block mainnet workload covering heights 3,220,327 through 3,470,326. Pre-NU6.3 Orchard compact actions and post-NU6.3 Ironwood compact actions are treated as one Orchard-

family workload for action density and byte-volume measurement. They are not presented as a comparison between the two pools.

14.1 Historical workload

Quantity	Observed value
Blocks	250,000
Orchard-family actions	688,370
Action-bearing transactions	276,271
Compact protobuf payload	143.14 MB
Mean actions per block	2.753
Payload per block, p50 / p95 / p99	448 / 1,876 / 3,427 bytes
Sequential acquisition time in retained capture	67.96 s

Matched endpoint samples later produced a median observed payload throughput of 0.892 MB/s and median request overhead of 265 ms. Applying that calibration to a six-month full-tail reacquisition gives 160.4 seconds at baseline traffic. This is a **modeled** acquisition value, not a service-level guarantee.

14.2 Wallet-owned position replay

Replay path	Time	Evidence class
Reference Coppice components with duplicate tree work	239.519 s	Measured
Production `CorePositionRuntime` plus Names	1.853 s	Measured
Independent tree-free confirmation consumer	1.706 s	Measured

The production path reduced measured Coppice component time by 99.23%. It matched every global action position and block tree-size transition, matched wallet/Core roots at 250 fixed checkpoints and the final tip, reached the same 688,370-leaf root, and reproduced the final state after rolling back and reapplying the last block.

This benchmark isolates in-memory replay. It excludes wallet SQLite cost, network transfer, routed full transactions, Names proof verification on live Coppice traffic, and a live multi-block replacement-branch reorganization. Those exclusions prevent treating 1.853 seconds as an end-to-end user latency.

14.3 Arbitrary-name lookup choices

At baseline traffic, the current model gives:

Evidence strategy	Six-month cost	Classification
Reacquire full compact tail remotely	160.4 s	Modeled
Fetch nullifier effects plus 218 separate name windows	61.2 s	Modeled
Traverse retained local compact evidence	0.224 s raw decode	Measured component
Production Core-position plus Names replay	1.853 s	Measured component

The sparse remote strategy downloads fewer bytes but pays round-trip overhead for many disjoint windows and creates more observable query behavior. The preferred path retains approximately 143 MB of rolling compact evidence in the measured baseline and resolves locally. A sparse nullifier journal plus `COMMIT` tail remains a lower-storage fallback, not the default fast path.

14.4 Synthetic adoption loads

Synthetic, non-consensus Names-shaped transactions were inserted into a copy of the historical workload at declared rates. A default synthetic transaction contains 13 Orchard-family actions, matching the qualified transport shape but not claiming to be a valid mainnet Coppice transaction.

Names tx/day	Six-month payload	Raw decode	Modeled acquisition
0	143.14 MB	0.224 s	160.4 s
10	147.71 MB	0.237 s	165.5 s
100	188.91 MB	0.311 s	211.7 s
1,000	600.82 MB	0.974 s	673.3 s

The experiment indicates that evidence acquisition, rather than protobuf decoding, dominates cold lookup across the modeled range. Synthetic results describe scaling behavior and are not adoption forecasts.

14.5 Adversarial load

Under an intentionally extreme 2 MB block-fill model, continuous generic-route acquisition could cause roughly 40.5 million unrelated full-transaction fetches, 499 GB of data, and 39.9 hours of local route processing over six months. The exact referenced-`COMMIT` design reduces unrelated generic full-transaction fetches during exact resolution to zero — this is the quantified version of the asymmetry in Figure 7.

A targeted attacker can instead fill one name's 24-block window. The current model bounds that window at approximately 1,080 reveal-shaped candidates and 21.34 seconds of route plus invalid-proof CPU, in addition to the attacker's need to publish referenced `COMMIT`s and pay Zcash fees. This is a conservative modeled ceiling, not a measured live attack.

14.6 Evaluation limits

Before deployment, the following measurements remain **WIP**:

- complete wallet lookup latency including SQLite and UI;
- proof generation and verification for actual V6 13-action Names transactions;
- live routed full-transaction acquisition;
- restart and cache-reconstruction latency for owned names;
- replacement-branch reorganization under managed note insertion; and
- multi-device and multi-provider availability behavior.

The retained corpus, deterministic synthetic generator, raw results, model, and charts form the current reproducibility package. The generator and model live in the zcash-devtool repository. A stable public archive of the retained corpus and a citation identifier are **TBD**.

15 Conformance and deployment

The checked-in positive conformance artifact freezes the deployment, verifier suite, route derivation, schedule, statements, real proofs, operation encoding, and resolved heads; its vector-set digest is recorded in the accompanying manifest. Rust and independent Python consumers verify the same artifact. Negative tests reject malformed encodings, wrong routes and networks, stale predecessors, invalid timing, wrong action indexes, proof substitution, wrong bond values, noncanonical order, and wrong-branch snapshots.

Conformance does not substitute for deployment qualification. Before a production activation, the project must finalize the production parameter set and verifier identities, publish reproducible artifacts, complete independent review, and exercise the complete lifecycle and reorganization behavior over a representative live environment. The production activation height is **TBD**.

16 Limitations and open work

Coppice Names deliberately accepts several limitations:

- Names and UAs are public after `REVEAL`.
- Exact remote queries can leak lookup interest unless evidence is acquired by ordinary wallet sync.
- A resolver must retain or reacquire canonical evidence; Zcash consensus does not provide a Names state commitment.
- A fixed 1 ZEC bond limits accessibility when ZEC's purchasing power changes.
- Updates and renewals wait for the name's daily window by design.
- Cooldown sacrifices immediate reuse to reduce abrupt impersonation risk.
- The protocol proves control, not legal or social identity.
- Name transfer is unsupported.

Open engineering and assurance work is listed as WIP rather than silently assumed complete. Future investigation may improve circuits, proof costs, storage layout, or acquisition privacy, but any change must preserve arbitrary resolution, canonical replay, hidden authority, and the absence of a trusted resolver.

17 Conclusion

Coppice Names turns the Zcash chain into a privacy-aware naming substrate without asking Zcash consensus to understand names. A refundable shielded bond represents continuing control; zero-knowledge proofs validate private ownership transitions; deterministic replay validates public protocol state; and name-derived routes make arbitrary resolution practical.

The principal performance result is architectural rather than merely an implementation optimization: wallets should retain canonical evidence during their ordinary scan and let Names reuse wallet-owned commitment-tree facts. That keeps resolution local, fast, and independently verifiable without promoting a third-party service or mutable local index into protocol authority.

Glossary

Action One Orchard/Ironwood shielded input–output pair, revealing only a nullifier and a note commitment.

Bond The exact 1 ZEC shielded note backing an active record. Owned by the registrant; spending it releases the name.

Bulletin A published application message, framed by CPCF inside a CAPP envelope and carried by zero-value notes.

CAPP Coppice's application envelope, naming one exact content-addressed application identity.

Carrier A zero-value note whose ciphertext carries bulletin bytes.

Cooldown One epoch of non-resolving quarantine after expiry or release.

CommitRef Canonical reference to one transaction: height, transaction index, transaction ID.

Deployment identifier Hash over the fixed deployment preimage (parameters, identities, semantic-ruleset fingerprint, verifiers); separates deployments cryptographically.

Designated action The Ironwood action that spends and creates bond notes, bound by the operation proof.

Epoch E consecutive canonical blocks starting at activation; each name has one operation window per epoch.

Exact resolver A resolver that replays one name's history and must agree with full replay at the same tip.

Head The currently accepted record state for a name: producer reference, UA, lease, and lineage.

Ironwood The post-NU6.3 Zcash shielded-pool transaction model that Coppice consumes.

Lease The interval an accepted head remains active; renewed in full by every accepted `REFRESH`.

NameId Nonzero Pallas field element derived from the canonical label.

Name route Public rendezvous route derived from deployment and name identifiers; carries `REVEAL` and `REFRESH`.

Name window The half-open block interval within an epoch in which the name's operations are accepted.

Nullifier The public spend marker of a shielded note; observing it proves the note is spent without revealing what was spent.

Reducer The deterministic state machine that applies authenticated Names operations in canonical order.

Snapshot Derived reducer state with rollback journals; never a consensus commitment.

StateRef Canonical reference to one action: height, transaction index, transaction ID, action index.

UA ZIP-316 Unified Address; the record's payment destination.

References

[1] The Electric Coin Company and Zcash contributors. *Zcash Protocol Specification*. `https://zips.z.cash/protocol/protocol.pdf`.

[2] Zcash Improvement Proposal 258. *Deployment of the NU6.3 Network Upgrade*. `https://zips.z.cash/zip-0258`.

[3] Zcash Improvement Proposal 229. *Version 6 Transaction Format*. `https://zips.z.cash/zip-0229`.

[4] Zcash Improvement Proposal 316. *Unified Addresses and Unified Viewing Keys*. `https://zips.z.cash/zip-0316`.

[5] Zcash Improvement Proposal 317. *Proportional Transfer Fee Mechanism*. `https://zips.z.cash/zip-0317`.

[6] Zcash Improvement Proposal 318. *Orchard to Ironwood Migration*. `https://zips.z.cash/zip-0318`.

[7] Zcash Improvement Proposal 326. *NU6.3 Consequences for Wallets*. `https://zips.z.cash/zip-0326`.

[8] The halo2 contributors. *Polynomial commitment using an inner product argument*. `https://zcash.github.io/halo2/background/pc-ipa.html`.

[9] The Coppice contributors. *Coppice: a deterministic application runtime over canonical Zcash history*. `https://github.com/nfl0/coppice`.

[10] The Namecoin project. *Namecoin documentation*. `https://www.namecoin.org/`.

[11] The ENS community. *Ethereum Name Service documentation*. `https://docs.ens.domains/`.

[12] The Handshake community. *Handshake protocol documentation*. `https://handshake.org/`.